

LECTURE 27

Case Study: Memory Management in Windows and ARM

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

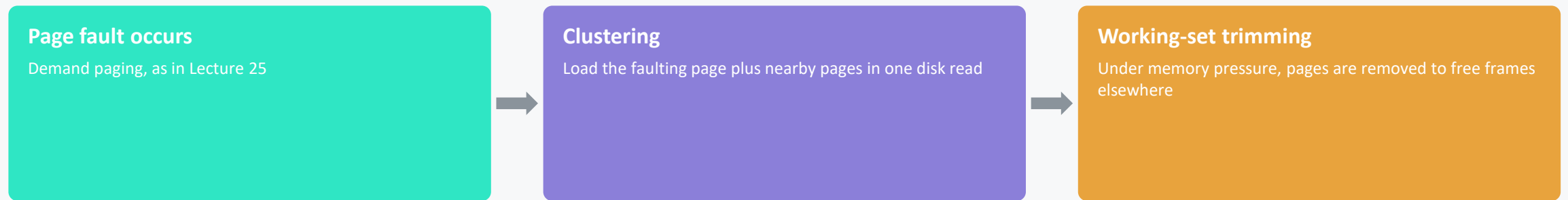
In This Lecture

- How Windows combines demand paging with **clustering** to exploit locality of reference
- The **working-set trimming** Windows performs under memory pressure
- The conceptual difference between **reserving** and **committing** virtual memory
- How ARM's MMU and page-table design reflect its dominance in memory-constrained devices
- **TrustZone**: ARM's hardware-enforced secure/normal world isolation

Memory Management in Windows

Windows uses demand paging as its baseline strategy, but it doesn't bring in just the one page that faulted — and on the eviction side, it manages memory per-process using a working-set approach.

Windows memory management, conceptually



Clustering is a bet on locality of reference. If a process just touched one page, it's likely to touch its neighbors soon too — one slightly larger disk read now is cheaper than several small ones later. Working-set trimming removes pages (typically ones not referenced recently) from processes to free frames for others that need them more urgently right now.

Reserving vs. Committing Virtual Memory

Windows exposes virtual memory to applications through an API built around a deliberate two-step distinction, mirroring Lecture 25's core idea: a large range costs nothing until it is actually used.

Reserving

Guarantees a range of virtual addresses won't be handed out to anything else — commits no physical memory at all



Committing

Actually guarantees physical memory (or backing page-file space) will be available when the range is touched

Why this matters. An application can claim room to grow — say, for a data structure that might expand significantly — without paying for memory it may never touch.

Memory Management on ARM

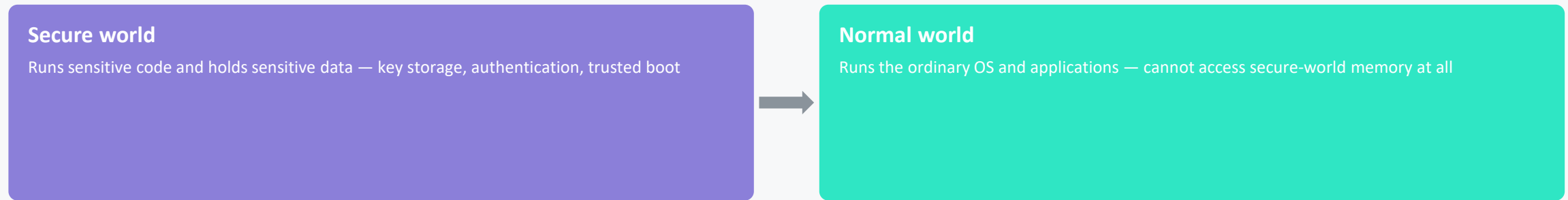
ARM's Memory Management Unit (MMU) performs the same job as the MMU in any paging system — translating virtual addresses to physical ones through page tables.

- ARM's widespread use in mobile and embedded devices, where memory is far more constrained than on a typical desktop, makes multi-level page tables even more essential than on x86.
- A flat, single-level page table sized for a large address space would itself consume memory an embedded device cannot spare.
- Breaking the table into multiple levels, where entire branches can be left unallocated until actually needed, keeps page-table overhead proportional to how much address space a process is genuinely using.

Same translation idea, tighter budget. Multiple page-table levels, a TLB to cache results, permission bits per page — the same conceptual job as x86, but every byte spent on page-table overhead is a byte not available for the applications phones, tablets, and embedded systems actually run.

TrustZone: Security Through Memory Isolation

TrustZone is an ARM-specific security feature that puts a hardware boundary directly inside the memory system, splitting a system into two worlds.



Enforced in hardware, not software. The isolation is enforced at the memory system itself — the same protection goal introduced earlier in this course, now raised to the hardware level, so that even a fully compromised normal-world OS still cannot read or tamper with whatever the secure world is protecting.

Key Takeaways

- **Windows** pairs demand paging with clustering (loading neighboring pages speculatively) and manages eviction through working-set trimming under memory pressure.
- **Reserving** an address range is free; **committing** it actually guarantees backing memory — letting applications claim room to grow without paying for it upfront.
- **ARM's MMU** performs the same translation role as any paging hardware, but its dominance in memory-constrained devices makes multi-level page tables even more essential than on desktop-class x86.
- **TrustZone** splits an ARM system into a secure world and a normal world, with hardware-enforced memory isolation between them.